

Design and Deployment of Lightweight Edge AI Models for IoT Network Intrusion Detection

Cylia Khelifi¹

¹Dept. of Computer Engineering, Institute of Electrical Engineering and Electronics,
Boumerdes Dept. of AI and Embedded Systems

October 30, 2025

Design and Deployment of Lightweight Edge AI Models for IoT Network Intrusion Detection

Cylia Khelifi

Dept. of Computer Engineering, Institute of Electrical Engineering and Electronics, Boumerdes

Dept. of AI and Embedded Systems, Qareeb

Algiers, Algeria

cylia.khelifi@qareeb.io

Abstract—The rapid expansion of the Internet of Things (IoT) has increased the need for efficient and secure network protection mechanisms that can operate under limited computational resources. Traditional Network Intrusion Detection Systems (NIDS) are often too complex for deployment on low-power edge devices. This work focuses on the design, training, and deployment of lightweight Artificial Intelligence (AI) models for IoT network intrusion detection at the edge. Several machine learning algorithms, including Decision Tree, Random Forest, Naïve Bayes, and Neural Network models, were trained and evaluated on CIC-IoT-2023 intrusion detection dataset. The models were deployed on embedded platforms, specifically the Raspberry Pi 4 and the ESP32 microcontroller, to evaluate real-time inference performance and resource utilization. Results demonstrate that lightweight AI models can achieve high detection accuracy and precision exceeding 95% while maintaining low latency and minimal memory footprint, making them suitable for practical edge-based intrusion detection in IoT environments.

Index Terms—Edge AI, Internet of Things (IoT), Artificial Intelligence, Network Intrusion Detection System (NIDS), CIC-IoT-2023, Machine Learning, Embedded Systems, Raspberry Pi, ESP32, Model Optimization, Edge Computing

I. INTRODUCTION

The Internet of Things (IoT) has become one of the most transformative technologies in modern computing, enabling continuous connectivity and seamless data exchange among devices across diverse applications. Despite its wide adoption, this interconnectivity has introduced critical security challenges. IoT devices are often limited in computational power, memory, and energy capacity, which prevents the implementation of conventional, resource-intensive security mechanisms. As a result, IoT infrastructures have become vulnerable to a wide range of cyber attacks that can disrupt services, compromise data, or even take control of devices [1].

Among the most effective mechanisms for addressing these threats are Network Intrusion Detection Systems (NIDS). These systems monitor network traffic to detect malicious or anomalous activity that could indicate an ongoing attack. However, traditional NIDS solutions typically rely on centralized, cloud-based analysis or high-performance computing environments, both of which are unsuitable for resource-constrained IoT devices. This limitation has motivated research into lightweight, decentralized intrusion detection systems that can operate directly at the network edge.

Edge computing offers an efficient alternative by shifting data processing closer to the source of data generation. When combined with Artificial Intelligence (AI), edge-based systems can analyze network traffic locally, reducing latency while preserving data privacy. This paradigm, known as Edge AI, has gained increasing attention for security-critical IoT applications.

This work focuses on the design, training, and deployment of lightweight AI models for IoT network intrusion detection at the edge. The goal is to create models that achieve a balance between detection accuracy and computational efficiency, allowing them to be executed on low-power devices such as the Raspberry Pi 4 and the ESP32 microcontroller. Several models were trained, including Decision Tree, Random Forest, Naïve Bayes, and Neural Network, on CIC-IoT-2023 intrusion detection dataset to classify network traffic as either normal or malicious, and to identify different categories of attacks.

The trained models were evaluated based on accuracy, precision, recall, F1-score. The deployment phase involved integrating and testing the models on the Raspberry Pi 4 and ESP32 platforms. The results demonstrate that ML models can achieve high intrusion detection accuracy while maintaining low inference time and minimal memory footprint on embedded platforms. This confirms the feasibility of deploying intelligent, real-time intrusion detection systems directly on edge device without relying on centralized cloud infrastructure.

II. THEORETICAL BACKGROUND

A. Cybersecurity and Intrusion Detection Systems (IDS)

Cybersecurity is the process of protecting networks, systems, data, and programs from unauthorized access or criminal exploitation, and it aims at ensuring the safety of digital and physical assets, and managing cyber risks that may arise [2].

1) *Types of Cybersecurity Threats*: Cybersecurity threats are diverse and continuously evolving. Some of the most common threats include [2]: Malware, Phishing, and Social Engineering. Cyber threats can also be categorized based on their consequences [3]:

- DoS/DDoS Attacks: Threats to system availability.
- Masquerading Attacks (e.g., ARP Spoofing, MitM): Threats to confidentiality and integrity.

- Reconnaissance Attacks (e.g., Port Scan, Vulnerability Scan): Threats to confidentiality.
- Web-based Attacks (e.g., SQL Injection, XSS): Threats to all three—confidentiality, integrity, and availability.

As network attacks increase in number and complexity, IDSs have become essential to most security infrastructures. IDSs monitor network traffic or system events to identify intrusions—any attempts to compromise confidentiality, integrity, or availability (CIA) or to bypass security mechanisms [4]. IDSs are categorized mainly as Network-based (NIDS) or Host-based (HIDS) systems [5].

Classification Based on Detection Method: IDSs typically use signature-based or anomaly-based methods [6]. Signature-based detection matches observed events with known attack signatures, whereas anomaly-based systems identify deviations from normal behavior [4].

B. Internet of Things (IoT)

The Internet of Things connects physical devices embedded with sensors, software, and communication modules to collect and exchange data. Despite its benefits, IoT's massive scale and limited resources make it highly vulnerable to attacks. Lack of standardization and multiple access points heighten exposure [7].

C. Machine Learning

Machine Learning (ML), a subset of Artificial Intelligence (AI), allows systems to learn from data and experiences without explicit programming [8]. ML models learn from examples to perform classification, regression, and prediction tasks.

1) Machine Learning Algorithms used in this work:

a) **Naive Bayes:** A probabilistic model based on Bayes' theorem:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \quad (1)$$

It assumes feature independence and is efficient for high-dimensional data [9].

b) **Decision Tree:** A decision tree is a supervised learning model that recursively partitions the dataset into subsets based on feature values, aiming to maximize the separation between classes. The selection of the attribute for splitting is guided by criteria such as Information Gain or Gini Impurity [10].

Information Gain measures the reduction in entropy achieved by partitioning the data on a given attribute:

$$IG(S, A) = Entropy(S) - \sum_{v \in A} \frac{|S_v|}{|S|} \cdot Entropy(S_v) \quad (2)$$

where S is the set of instances, A is the attribute considered for splitting, and S_v is the subset of S for which attribute A has value v .

Entropy quantifies the impurity or randomness in a dataset:

$$H(S) = - \sum_{i=1}^n p_i \log_2(p_i) \quad (3)$$

where p_i is the proportion of instances in class i among n classes.

The Gini Index is another impurity measure, representing the likelihood of misclassification for a randomly chosen element:

$$Gini(S) = 1 - \sum_{i=1}^n p_i^2 \quad (4)$$

Lower Gini values indicate better class separation. Decision trees recursively split the data until a stopping criterion is met (e.g., maximum depth, minimum samples per leaf).

c) **Random Forest:** A Random Forest is an ensemble of decision trees that combines the predictions of multiple trees to improve accuracy and reduce overfitting [11]. Each tree is trained on a random bootstrap sample of the dataset, and at each split, a random subset of features is considered. Predictions are aggregated by majority voting for classification tasks or averaging for regression tasks. This ensemble approach mitigates the variance of single decision trees while maintaining robustness.

d) **Neural Networks:** Neural networks are inspired by biological neurons and model complex relationships through layers of weighted connections [12]. A single neuron (perceptron) computes its output as:

$$y = \phi \left(\sum_{i=1}^n w_i x_i + b \right) \quad (5)$$

where ϕ is the activation function, w_i are the weights, x_i are the input features, and b is the bias [12].

During training, the network learns by minimizing a loss function, such as the Mean Squared Error (MSE):

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m \left(h_w(x^{(i)}) - y^{(i)} \right)^2 \quad (6)$$

where m is the number of training examples, $x^{(i)}$ is the input of the i -th example, $y^{(i)}$ is the true output, and $h_w(x^{(i)})$ is the predicted output given weights w and biases b .

The weights and biases are then updated using gradient descent to minimize the loss:

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial J}{\partial w} \quad (7)$$

$$b_{\text{new}} = b_{\text{old}} - \alpha \frac{\partial J}{\partial b} \quad (8)$$

where α is the learning rate controlling the step size during optimization.

2) **Evaluation Metrics:** The performance of Machine Learning models is commonly evaluated using a confusion matrix, which is composed of the following elements:

- **True Positives (TP):** Correctly predicted attacks instances.
- **True Negatives (TN):** Correctly predicted normal traffic.
- **False Positives (FP):** Normal traffic incorrectly classified as an attack (false alarm).
- **False Negatives (FN):** Attacks incorrectly classified as normal traffic (missed detection).

From these quantities, several metrics are derived to assess model performance:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (9)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (10)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (11)$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (12)$$

D. Implementation Tools and Technologies

1) *Raspberry Pi*: A single-board computer developed by the Raspberry Pi Foundation for accessible computing. Models like the Pi 4 feature ARM Cortex-A72 CPU, up to 8GB RAM, and dual 4K HDMI outputs.

2) *ESP32 and ESP-IDF*: ESP32 is a Wi-Fi/Bluetooth-enabled microcontroller by Espressif Systems. It includes multiple peripherals and a dual-core Xtensa CPU. The ESP-IDF is the official SDK for ESP32 devices.

III. SOFTWARE DESIGN

This chapter presents the software design and implementation stages of the proposed intelligent intrusion detection system. The section covers model inference, dataset selection, preprocessing, model building, and deployment on both Raspberry Pi and ESP32 platforms. The focus is on the artificial intelligence pipeline and its deployment on embedded devices for real-time operation.

A. ML Model Development

The development process of the machine learning model involves selecting an appropriate dataset, performing data preprocessing, selecting algorithms, and building the final models for comparison and deployment.

1) *Dataset Selection and Preprocessing*: Numerous NIDS datasets exist, each targeting different network scenarios. Intrusion detection datasets are generally constructed by capturing and labeling network traffic flows containing both benign and malicious packets. Flow analysis is performed by examining various packet-level and flow-level attributes such as source and destination IPs, ports, protocol types, and transport-layer flags [13].

Several popular datasets used in this domain include:

- **NSL-KDD**: A refined version of KDD'99 with four attack types (DoS, Probe, R2L, U2R).
- **UNSW-NB15**: Includes 49 features and nine attack categories such as DoS, Exploits, Backdoors, and Worms.
- **IoT-23**: Focuses on IoT network protocols (MQTT, HTTP) and distinguishes benign from malicious traffic.
- **Bot-IoT**: Contains both normal and botnet traffic, including DDoS, DoS, and keylogging attacks.
- **CIC-IDS-2017**: Labeled network flows provided by the Canadian Institute for Cybersecurity.
- **CIC-IoT-2023**: Represents modern IoT environments with 33 attack types categorized into seven families.

a) *CIC-IoT-2023 Dataset*: The CIC-IoT-2023 dataset was selected for this work due to its relevance to IoT systems and its wide range of up-to-date attack types [14]. It includes 33 attacks across 105 devices, grouped into seven major categories: DDoS, DoS, Reconnaissance, Web-based, Brute Force, Spoofing, and Mirai.

TABLE I: Dataset Features

Feature	Description
Destination IP Address	IP address of the destination device.
Destination Port	Port number of the destination device.
Source IP Address	IP address of the source device.
Source Port	Port number of the source device.
Protocol Type	Network protocol used (UDP, TCP, IGMP, ICMP).
Flag	TCP flag values (e.g., FIN, SYN, ACK, RST, URG).
ACK Count	Number of packets with ACK flag set in the same flow.
SYN Count	Number of packets with SYN flag set in the same flow.
FIN Count	Number of packets with FIN flag set in the same flow.
URG Count	Number of packets with URG flag set in the same flow.
RST Count	Number of packets with RST flag set in the same flow.
Transport Layer Protocol	Transport layer protocol (TCP or UDP).
Application Layer Protocol	Application layer protocol (HTTP, DNS, Telnet, etc.).
Rate	Overall packet transmission rate in a flow.
Srate	Outbound packet transmission rate in a flow.
Drate	Inbound packet transmission rate in a flow.
Tot Sum	Total sum of packet lengths in a flow.
Min	Minimum packet length in the flow.
Max	Maximum packet length in the flow.
AVG	Average packet length in the flow.
Std	Standard deviation of packet lengths in the flow.
Tot Size	Individual packet length.
Number	Total number of packets in the flow.
Flow Bytes/s	Flow rate in bytes per second.
Flow Packets/s	Flow rate in packets per second.
Magnitude	$\sqrt{(\text{Avg_in}^2 + \text{Avg_out}^2)}$ — magnitude of average incoming/outgoing lengths.
Radius	$\sqrt{(\text{Var_in}^2 + \text{Var_out}^2)}$ — radius of variances of incoming/outgoing lengths.

Continued on next page

Continued from previous page

Feature	Description
Covariance	Covariance of lengths of incoming and outgoing packets.
Variance	Ratio of variances of incoming to outgoing packet lengths.
Weight	Product of number of incoming and outgoing packets.

Table I presents a detailed description of a set of features present in the dataset. The ‘Timestamp’ records the specific time each packet is captured. ‘Flow Duration’ shows how long a packet has been in transit. ‘Protocol Type’ classifies packets by their network protocols, including common ones like IP, UDP, and TCP. The dataset also includes indicators for application layer protocols such as ‘HTTP’, ‘HTTPS’, and ‘DNS’ which allows to identify specific application-level behaviors. Counts of flags like ‘FIN’, ‘SYN’, ‘RST’, ‘ACK’, and ‘URG’ provide information on specific packet-level interactions and potential anomalies. Statistical metrics like ‘Covariance’ and ‘Variance Ratio’ measure the variability in packet lengths. ‘Weight’ represents the total count of incoming and outgoing packets and offers a comprehensive view of traffic patterns. Several features seen above are considered categorical data; ‘Protocol Type’ feature in the dataset is a categorical one that indicates the type of network protocol: TCP or UDP. To handle categorical features, one-hot encoding is employed. It is a technique used to convert categorical variables into a numerical format suitable for ML algorithms.

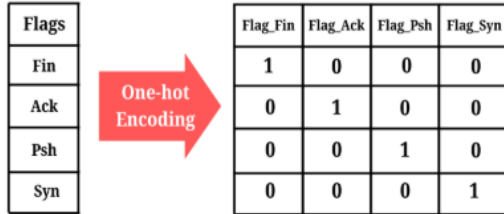


Fig. 1: One-hot Encoding

The dataset originally contained over 40 million unbalanced samples, leading to long training time. Therefore, undersampling was applied to the majority classes (e.g., DoS, DDoS) to balance the data distribution.

2) *ML Model Building*: Several algorithms were explored to identify the most suitable model for deployment.

a) *Naive Bayes*: Calculates class priors and feature likelihoods, using log-probabilities to prevent underflow during computation. The trained model predicts the class of a new instance based on posterior probabilities.

b) *Decision Tree*: Constructed with a maximum depth of 50 using Gini Impurity as the splitting criterion. Once trained, the model is exported into a lightweight .pkl file, later converted into an equivalent C-based decision function for embedded inference.

c) *Random Forest*: Comprises 50 decision trees, combining their predictions via majority voting to reduce overfitting and variance.

d) *Neural Network*: A feedforward Artificial Neural Network (ANN) was designed and trained to perform intrusion detection using packet and flow-based features. The model consists of four fully connected (dense) layers, with the first layer containing 64 neurons, followed by 32, 16, and 8 neurons respectively. The weights in all layers were initialized using the Glorot Uniform initializer to ensure stable convergence during training.

The ReLU activation function is applied to each hidden layer to introduce non-linearity and accelerate training:

$$\text{ReLU}(x) = \max(0, x) \quad (13)$$

The output layer uses the Softmax activation function to produce class probabilities across multiple attack categories:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (14)$$

The model is optimized using the Adam optimizer, minimizing the categorical cross-entropy loss defined as:

$$\text{Loss} = - \sum_i y_i \log(p_i) \quad (15)$$

Batch normalization was applied after each dense layer to stabilize and accelerate the learning process, with parameters initialized as $\gamma = 1$, $\beta = 0$, and moving mean and variance set to 0 and 1, respectively.

B. Raspberry Pi Implementation

The Raspberry Pi platform was responsible for both data preprocessing and model inference operations. All scripts were developed in Python for packet parsing, feature extraction, and machine learning. The libraries used were dpkt, scapy, pandas, tqdm, and TensorFlow.

Network traffic data was imported from previously captured PCAP files, parsed, and converted into structured feature vectors to be fed to the ML models. Each packet was decoded to extract low-level attributes such as source and destination IP addresses, ports, protocol type (TCP, UDP, ICMP), and TCP control flags (e.g., SYN, ACK, FIN, RST). Packets sharing the same 5-tuple (source IP, destination IP, source port, destination port, and protocol) were aggregated into bidirectional flows to represent communication sessions. For each flow, statistical and temporal metrics including packet counts, flow duration, transmission rates (bytes/s and packets/s), and size distribution (minimum, maximum, mean, and standard deviation, magnitude, radius, covariance) were computed. Once the features were extracted, a function—generated from trained models including Decision Tree, and Random Forest variants—executed the inference step to classify network traffic in real time. For classical machine learning models, the function was automatically translated into optimized if-else logic, while TensorFlow-based models were executed using the TFLite interpreter for efficiency on ARM architecture.

To maintain processing efficiency, a multi-threaded approach was adopted:

- **Thread 1:** Feature Extraction and Preprocessing
- **Thread 2:** Model Inference

This configuration allowed the Raspberry Pi to perform processing, from raw packet analysis to classification, demonstrating its capability as an effective edge computing node for intrusion detection.

C. ESP32 Inference Deployment

In this work, the ESP32 microcontroller was used exclusively for inference to evaluate the feasibility of running the trained models on a low-power embedded device. Although the ESP32 is capable of performing the data processing part, in this implementation, preprocessing was intentionally executed externally. This approach was adopted to test the reliability and timing of I²C communication, ensuring that preprocessed feature vectors could be correctly received and classified by the microcontroller.

Since multiple models were developed during this work, two inference approaches were implemented within the same project:

- **Predict_ML_Model:** Contains C implementations of classical machine learning models such as Decision Trees. These models were translated into nested `if-else` statements, allowing lightweight inference without requiring any additional library dependencies.
- **Predict_NN_Model:** Implements neural network inference using TensorFlow Lite for Microcontrollers (TFLite-Micro). The compiled model is stored as a binary array within `model_buffer.h`, which is read at runtime for performing classification.

Two FreeRTOS tasks were defined to enable concurrent operation:

- **I²C Communication Handler Task:** Receives preprocessed feature vectors over the I²C bus and forwards them to the inference queue.
- **Prediction Task:** Executes inference using either the ML-based or NN-based function, depending on the active configuration.

The final project directory structure is as follows:

```
model_inference/
main/
  main.c
  model_buffer.h
  predict_ml_model.c
  predict_ml_model.h
  predict_nn_model.c
  predict_nn_model.h
  CMakeLists.txt
```

This configuration validates that the ESP32 can successfully execute both classical and neural network inference tasks while maintaining reliable I²C communication.

IV. RESULTS AND ANALYSIS

In this paper, four ML models were trained to detect network intrusions in IoT networks. The objective of this analysis is to evaluate the performance of these models, covering their strengths and weaknesses within the context of intrusions.

A. Hypothesis on the Performance of ML Models

Among the four ML models trained for the Network Intrusion Detection System, it is hypothesized that their overall performance, ranked from the least to the most effective, will follow this order: Naive Bayes, Neural Network, Decision Tree, then Random Forest. Random Forest is expected to outperform other models due to its ensemble nature and robustness, followed by Decision Tree for its interpretability and efficiency. Neural Networks, while powerful, are hypothesized to rank third due to their resource demands. Naive Bayes is expected to have the lowest performance because of its strong independence assumptions, which may not represent the complex environment of IoT network traffic.

B. ML Models Evaluation

We evaluated each model using accuracy, precision, recall, and F1 score.

1) *Naive Bayes:* Naive Bayes was selected for its simplicity and ease of implementation.

TABLE II: Confusion Matrix Naive Bayes

Class	Accuracy	Precision	Recall	F1 Score
DDoS	0.8707	0.4776	0.9672	0.6392
DoS	0.8750	0.0000	0.0000	0.0000
Benign Traffic	0.7489	0.3351	0.5685	0.4214
Mirai	0.9028	0.9755	0.3287	0.4902
Spoofing	0.8611	0.6824	0.0886	0.1575
Reconnaissance	0.7296	0.2918	0.8650	0.4372
Web Based	0.8742	0.0000	0.0000	0.0000
Brute Force	0.8722	0.2291	0.0482	0.0793
Weighted Average	0.8475	0.4039	0.7111	0.4897

2) *Neural Network:* Neural Networks were selected for their ability to learn patterns from raw data and discover hidden structures.

TABLE III: Confusion Matrix Neural Network

Class	Accuracy	Precision	Recall	F1 Score
DDoS	0.7269	0.9999	0.0005	0.0011
DoS	0.9997	0.0000	0.0000	0.0000
Benign Traffic	0.5069	0.2150	0.9851	0.3524
Mirai	0.8450	0.5528	0.1998	0.2930
Spoofing	0.7063	0.9054	0.0017	0.0034
Reconnaissance	0.6855	0.1620	0.0218	0.0387
Web Based	0.6998	0.0000	0.0000	0.0000
Brute Force	0.6998	0.0000	0.0000	0.0000
Weighted Average	0.7321	0.2753	0.1088	0.0708

3) *Decision Tree:* Decision Trees were chosen for their simplicity and ability to handle non-linear relationships.

TABLE IV: Confusion Matrix Decision Tree

Class	Accuracy	Precision	Recall	F1 Score
DDoS	0.9999	0.9982	1.0000	0.9991
DoS	0.9999	0.9998	0.9999	0.9999
Benign Traffic	0.9967	0.7776	0.9937	0.8738
Mirai	0.9998	0.9999	1.0000	1.0000
Spoofing	0.9986	0.9460	0.9400	0.9430
Reconnaissance	0.9974	0.9085	0.9144	0.9114
Web Based	0.9985	0.9577	0.9021	0.9292
Brute Force	0.9935	0.9490	0.7759	0.8510
Weighted Average	0.9976	0.9517	0.9383	0.9425

4) *Random Forest*: Random Forest reduces overfitting by averaging results of multiple decision trees, providing robustness to noise.

TABLE V: Confusion Matrix Random Forest

Class	Accuracy	Precision	Recall	F1 Score
DDoS	0.9999	0.9884	0.9999	0.9941
DoS	0.9998	0.9998	0.9993	0.9996
Benign	0.9513	0.7514	0.9968	0.8559
Mirai	0.9999	0.9994	0.9971	0.9983
Spoofing	0.9912	0.9665	0.7713	0.8580
Reconnaissance	0.9817	0.8572	0.9232	0.8891
Web Based	0.9886	0.9354	0.8973	0.9160
Brute Force	0.9968	0.9972	0.8244	0.9034
Weighted Average	0.9910	0.9488	0.9240	0.9329

C. ML Models Comparison

Beginning with **Naive Bayes**, the model demonstrated moderate accuracy of 0.8475. However, its precision and recall varied across classes. It achieved relatively high recall for DDoS attacks but suffered from low precision, indicating a tendency to misclassify benign traffic as malicious. Moving to the **Neural Network**, its overall accuracy was lower at 0.7321. Although it achieved high precision for certain classes such as DDoS and Spoofing, recall remained low across multiple categories. As noted by Gaël Varoquaux, Research Director at Inria and co-creator of `scikit-learn`, tree-based models tend to achieve consistently higher scores than neural networks given equivalent training time [16]. In contrast, the **Decision Tree** model exhibited exceptional performance, achieving an accuracy of 0.9976. It maintained near-perfect precision and recall across nearly all classes, demonstrating robustness in detecting diverse intrusion types and a strong capability to generalize. Similarly, the **Random Forest** model achieved a high weighted average accuracy of 0.9910. Its precision and recall values closely matched those of the Decision Tree, confirming its stability and reliability in classifying both benign and malicious traffic. The ensemble nature of the Random Forest helps mitigate overfitting, offering a balanced compromise between model complexity and predictive power.

Categorical variables pose a well-known challenge for Neural Networks when applied to tabular datasets. Research by Varoquaux, Grinsztajn, and Oyallon [16] demonstrated that when training is restricted to purely numerical features, the performance gap between tree-based models and neural networks narrows considerably, suggesting that feature encoding significantly affects neural model efficiency. Furthermore, results from Enchun Shao's work [17] similarly confirmed that Decision Tree and Random Forest classifiers outperform Support Vector Machines (SVM) across multiple intrusion detection categories, reinforcing the suitability of tree-based architectures for network security applications.

D. Testing with Real Network Traffic

For a more concrete evaluation of the model, benign network traffic was captured in real time by sniffing live packets using `tshark`.

TABLE VI: Average Accuracy of Each Model

Model	Accuracy	Precision	Recall	F1 Score
Naive Bayes	0.8475	0.4039	0.7111	0.4897
Neural Network	0.7321	0.2753	0.1088	0.0708
Decision Tree	0.9976	0.9517	0.9383	0.9425
Random Forest	0.9910	0.9488	0.9240	0.9329

Testing on these real-time packets confirmed that the models maintain high performance in practical scenarios, reliably identifying normal traffic patterns without misclassifications.

V. CONCLUSION

This paper presents an AI-driven Network Intrusion Detection optimized for edge deployment in IoT networks. By processing data locally, the system ensures low-latency responses while preserving user privacy. Four Machine Learning algorithms were evaluated, with tree-based models achieving the highest accuracy, reaching 94.25% F1-Score, demonstrating robust detection across diverse intrusion types. Neural Networks and Naive Bayes showed limitations in generalization and consistency.

Future work will explore more advanced AI architectures, including CNNs, LSTMs, and hybrid models, to further improve detection capabilities, and extend edge-deployment of these models on diverse hardware platforms, such as STM microcontrollers, to enhance scalability and demonstrate the versatility of edge AI for cybersecurity in resource-constrained IoT environments.

REFERENCES

- [1] P. A. Networks, "98 percent of IoT traffic is unencrypted," [Online]. Available: <https://unit42.paloaltonetworks.com/iot-threat-report-2020/>
- [2] Cisco, "What is cybersecurity?" [Online]. Available: <https://www.cisco.com/c/en/us/products/security/what-is-cybersecurity.html>
- [3] L. B. William Stallings, *Computer Security: Principles and Practice*, PEARSON, 2014.
- [4] R. Bace and P. Mell, *Intrusion Detection Systems*, NIST National Institute of Standards and Technology, 2001.
- [5] ReasonLabs, "What is intrusion?" [Online]. Available: <https://cyberpedia.reasonlabs.com/EN/intrusion.htm>
- [6] A. Nasaf, "Intrusion detection system: A survey and taxonomy," HAL Open Science, 2021.
- [7] M. S. Aliero, A. M. Ahmad, U. S. Kalgo, and S. A. Aliero, "An overview of Internet of Things: Understanding the issues and challenges of a more connected world," *International Journal of Computing and Communication Networks*, vol. 2, no. 1, pp. 1–11, 2020.
- [8] A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM Journal of Research and Development*, 1959.
- [9] H. Zhang, "The optimality of Naive Bayes," ResearchGate, January 2004.
- [10] O. M. Lior Rokach, "Decision trees," ResearchGate, January 2005.
- [11] L. Breiman, "Random forests," ResearchGate, October 2001.
- [12] Y. B. Ian Goodfellow and A. Courville, *Deep Learning*, The MIT Press, 2016.
- [13] R. L. Ankit Thakkar, "A review of the advancement in intrusion detection datasets," ScienceDirect, 2020.
- [14] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," Sensor (2023).
- [15] UNSW Sydney - The University of New South Wales, "The Bot-IoT Dataset," 2021. [Online]. Available: <https://research.unsw.edu.au/projects/bot-iot-dataset>
- [16] G. V. Leo Grinsztajn, Edouard Oyallon, "Why do tree-based models still outperform deep learning on tabular data?" *Journal of Edge Computing*, July 2022.
- [17] E. Shao, "Encoding IP Address as a Feature for Network Intrusion Detection," Master's Thesis, Faculty of Purdue University, 2019.